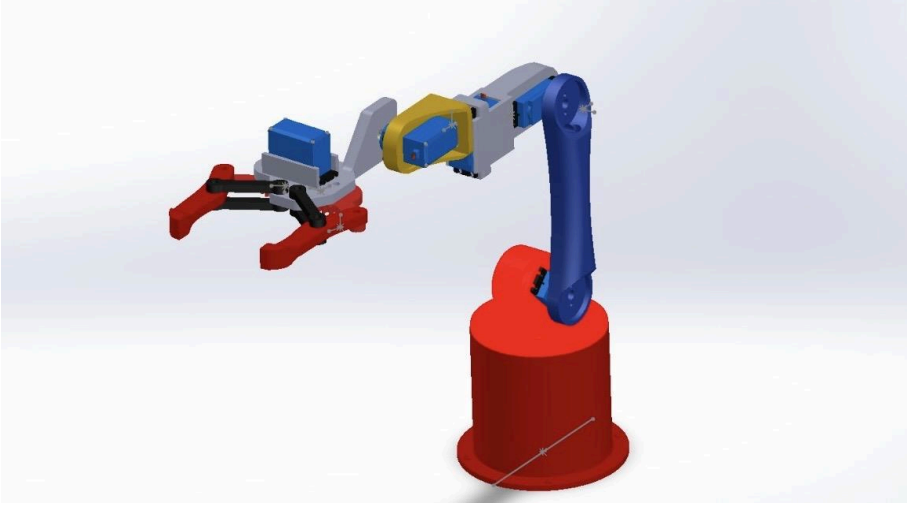
MCT342 Industrial Robotics

Milestone 1



Name	Student Code
Abdelrahman Farag	23p0137
Ahmad Hatem Mahmoud	23p0322
Basem Mohamed Fotoh	23p0005
Juliana Hany Saad ElDeen	23p0267
Mohamed Tarek Mohamed Elkholy	23p0142

Summary 2
Objectives 3
Robot Modeling 3
Simulation in Gazebo Sim 4
Package Structure and Organization 6
Challenges and Lessons Learned 7
Conclusion 7

Summary

In this milestone, our team successfully designed and simulated a 6-DOF robotic manipulator consisting of a 4-DOF robotic arm and a 2-DOF gripper using ROS 2. The primary objective was to create a virtual model that serves as a foundation for physical prototyping, ensuring realistic dimensions, motion, and actuation. We developed a complete URDF/Xacro model, integrated it into Gazebo Sim for dynamic simulation, and configured ROS controllers for basic actuation. Additionally, we implemented optional RViz2 visualization for enhanced testing. Key achievements include verifying joint kinematics, physics interactions, and real-world feasibility. Challenges, such as tuning joint limits for stability, were addressed through iterative simulation. This work complies with all requirements and sets the stage for future planning and control modules.

Objectives

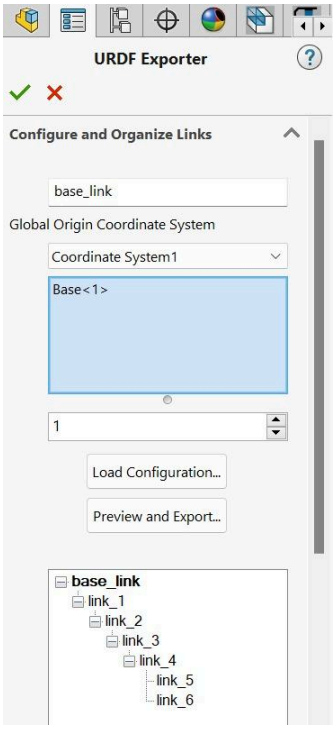
The milestone aimed to design and simulate a robotic system with a 4-DOF robotic arm and a 2-DOF gripper in ROS 2, focusing on visualization and basic actuation. The goal was to build a solid foundation for integration with planning and control modules, while ensuring the design is ready for real-world fabrication and testing.

Robot Modeling

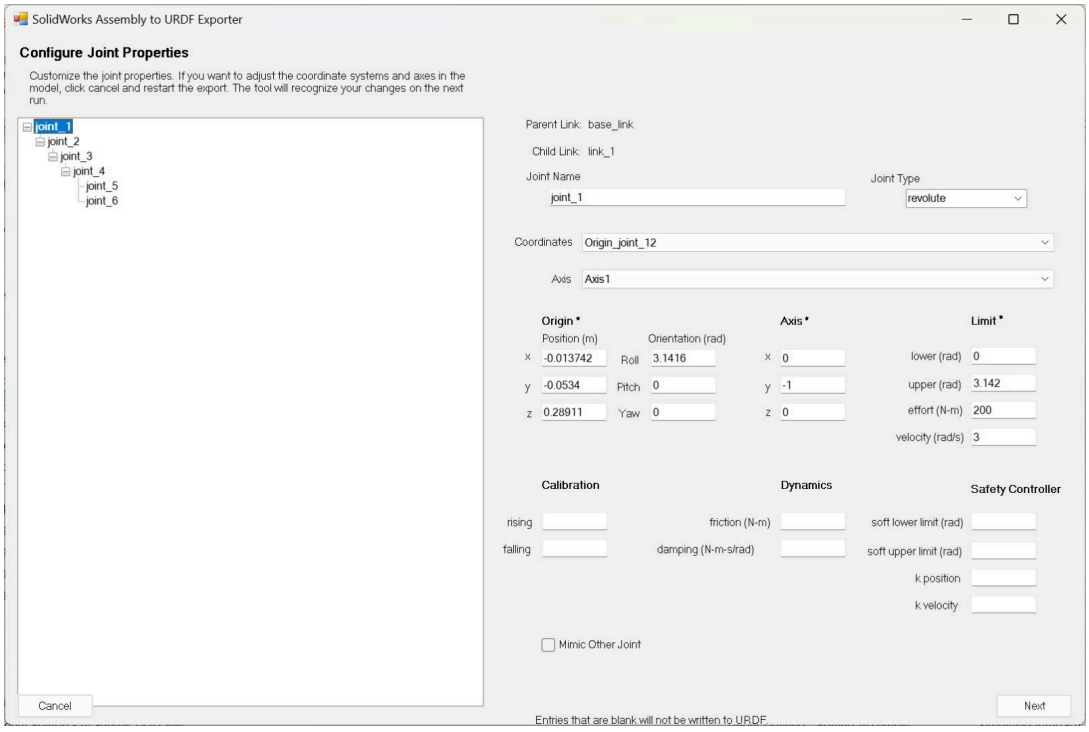
We began by creating a URDF (Unified Robot Description Format) or Xacro (XML Macros) model for the robotic system. The model includes a 4-DOF robotic arm with revolute joints for realistic motion, such as reaching and positioning, and a 2-DOF gripper attached as the end-effector for grasping tasks. We designed the arm with dimensions suitable for physical building, ensuring joint limits prevent singularities and allow full range of motion. For the gripper, we implemented two independently actuated fingers using prismatic joints for opening and closing, with a maximum aperture of 0.1 meters.

The URDF/Xacro file defines all link and joint properties comprehensively. Links include visuals using primitive shapes like cylinders and boxes for simplicity, or meshes if downloaded from GrabCAD. Collision properties were set with bounding boxes matching visuals to enable accurate physics simulation, while inertia properties were calculated based on material assumptions for stability. Joints are defined with types, limits (e.g., -180° to 180° for revolute joints), and

parent-child relationships. Coordinate frames and joint origins were specified using standard ROS conventions, with origins at joint centers for precise transformations.




```
<link name="base_link">
  <inertial>
    <origin xyz="6.2849E-10 2.346E-09 0.049723" rpy="0 0 0" />
    <mass value="0.1986" />
    <inertia ixx="0.00034708" ixy="5.6856E-11" ixz="-9.5919E-13" iyy="0.00034696" iyz="7.1178E-12" izz="0.00040
  </inertial>
  <visual>
    <origin xyz="0 0 0" rpy="0 0 0" />
    <geometry>
      <mesh filename="file:///home/ahmad_m7ey4b8ahmadhatem/ms_ws/src/ms1_pkg/meshes/base_link.STL" />
    </geometry>
    <material name="">
      <color rgba="0.79216 0.81961 0.93333 1" />
    </material>
  </visual>
  <collision>
    <origin xyz="0 0 0" rpy="0 0 0" />
    <geometry>
      <box size="0.1 0.1 0.1" />
    </geometry>
  </collision>
</link>
```

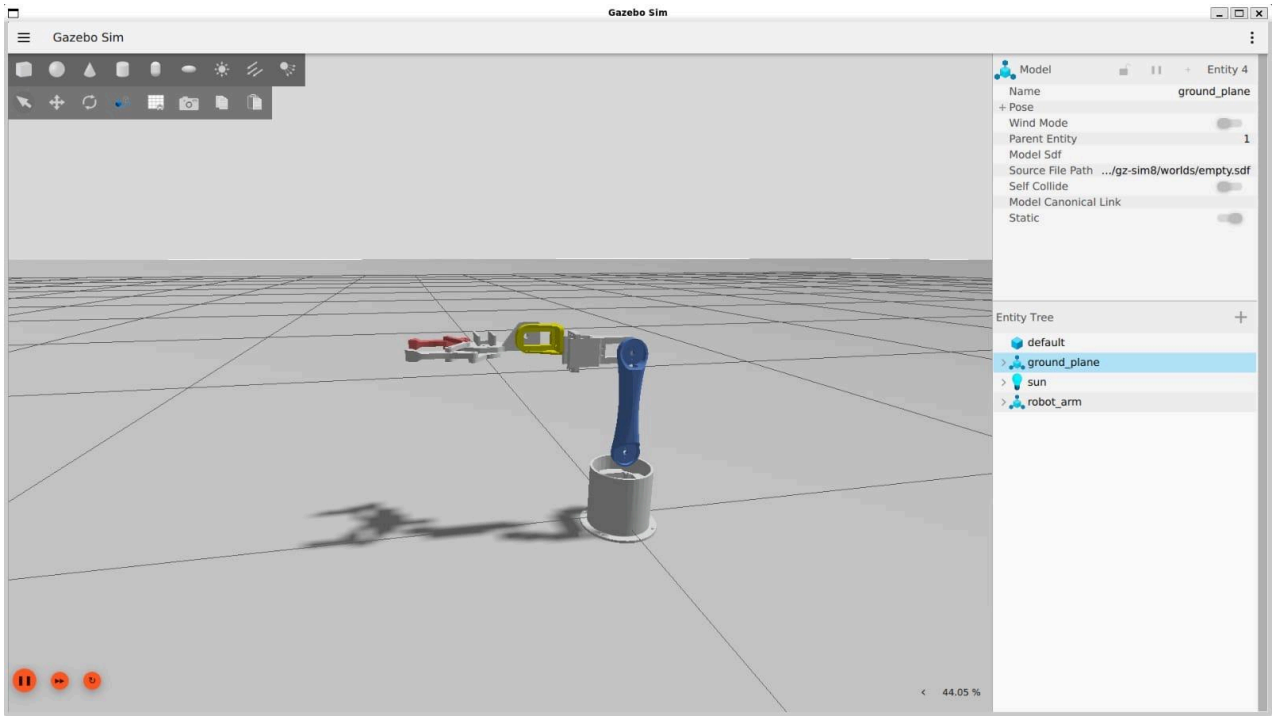


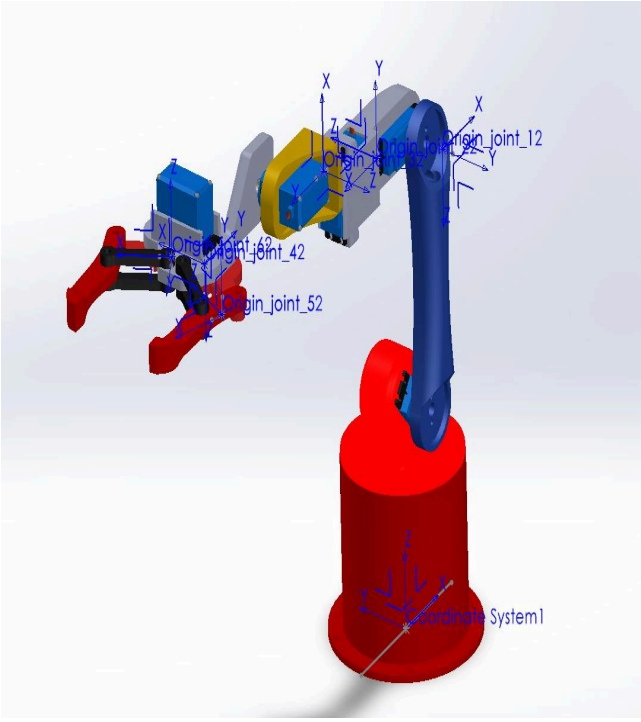
Simulation in Gazebo Sim

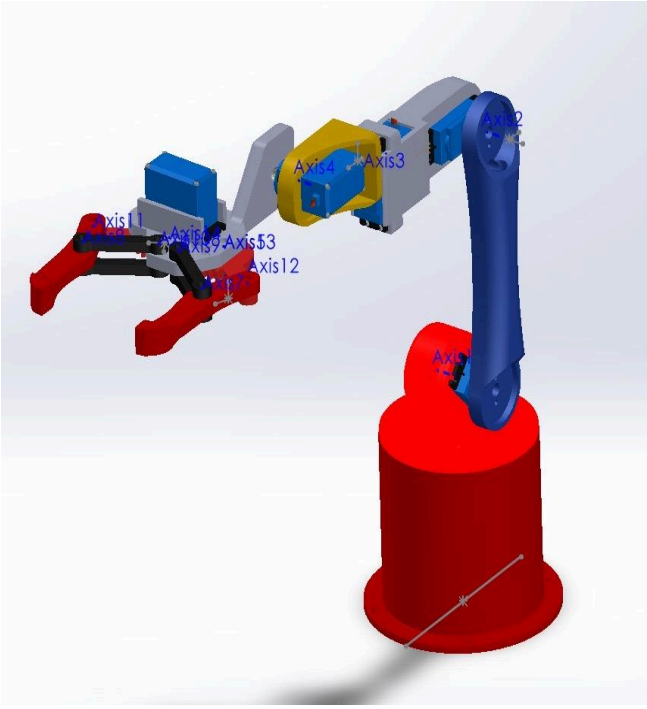
To enable dynamic simulation, we integrated the URDF/Xacro model into Gazebo Sim, a physics engine compatible with ROS 2. This involved adapting the model for Gazebo by adding plugins for ROS controllers, allowing joint actuation and sensor feedback. We configured ROS controllers using a .yaml file, which defines parameters such as joint controllers (e.g., position controllers for each of the 6 DOF), PID gains for stability (e.g., proportional gain of 100, integral of 0.1), and mappings to hardware interfaces. The .yaml file ensures smooth control during simulation, with entries for each joint like "joint1_controller" specifying type and parameters.

In addition, we created a .launch.py file to automate the simulation launch, including loading the world, spawning the robot, and starting controllers. The launch file uses ROS 2 launch actions to include Gazebo, load the URDF, and activate nodes like joint_state_publisher.

Moreover, in Gazebo Sim, we verified the robot's behavior through dynamic tests, such as applying forces or commanding joint movements. The simulation confirmed realistic physics, including gravity effects on the arm and collision detection with the environment.







Package Structure and Organization

The robot package follows ROS 2 conventions, with a structured directory layout including folders for launch/, urdf/, config/, and meshes/. Key files include the URDF/Xacro, .launch.py for Gazebo, .yaml for controllers, and optional RViz2 config files. The RAR file submitted contains all these elements, organized for easy reproduction.

ROS 2 Package Structure (`ms1_pkg`)

```
ms1_pkg/
├── CMakeLists.txt
├── package.xml
├── launch/
│   └── sim.launch.py
├── urdf/
│   └── my_robot_urdf.urdf
├── meshes/
│   ├── base_link.STL
│   ├── link_1.STL
│   └── ... (all other STL files)
├── config/
│   └── joint_controller.yaml
└── rviz/
    └── my_robot.rviz
```

Challenges and Lessons Learned

During modeling, defining accurate inertia for dynamic stability was challenging; we resolved it by referencing standard robotic arm specs. In simulation, initial joint oscillations were fixed by adjusting PID gains in the .yaml file. This milestone taught us the importance of iterative testing for real-world alignment.

Conclusion

In conclusion, we have fully met the milestone requirements, delivering a functional 6-DOF manipulator simulation ready for physical implementation. The design, simulation, and visualization ensure correct motion and actuation, paving the way for advanced modules.